SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

School of Computing

Tiruchirappalli, Tamil Nadu - 621 105

Department of Computer Applications

PRACTICAL RECORD

NAME :

REGISTER NO. : RA25322410500

PROGRAMME : MCA

SPECIALIZATION : GENERAL

SEMESTER/ YEAR : II / I

COURSE CODE : PCA25C06J

COURSE NAME : ADVANCED OPERATING SYSTEM

April 2026

SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

School of Computing

Tiruchirappalli, Tamil Nadu - 621 105

Department of Computer Applications

BONAFIDE CERTIFICATE

This is to certify that the bonafide work is done by Mr/Ms. _____ Register No. RA25322410500 in **Advanced Operating System(Course Code: PCA25C06J)** at Computer Lab, SRMIST, Tiruchirappalli, in April 2026.

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for the University Practical Examination held at SRMIST, Tiruchirappalli, Department of Computer Applications on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDEX

S. No.	Date	Name of the Program	Page No.	Signature
1	02.12.2025	SIMULATION OF PROCESS SCHEDULING	04	
2	09.12.2025	IMPLEMENTATION OF SYSTEM CALLS	07	
3	16.12.2025	IMPLEMENTATION OF DEADLOCKS	10	
4	23.12.2025	IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHM	13	
5	06.01.2026	IMPLEMENTATION OF SCHEDULING ALGORITHM	16	
6	13.01.2026	IMPLEMENTATION OF FILE ACCESS METHODS	19	
7	20.01.2026	IMPLEMENTATION OF DISTRIBUTED SCHEDULING	22	
8	27.01.2026	IMPLEMENTATION OF CLOCK SYNCHRONIZATION	25	
9	03.02.2026	IMPLEMENTATION OF CONCURRENCY CONTROL ALGORITHMS	28	
10	10.02.2026	WRITE A PROGRAM USING POSIX REAL-TIME THREADS	31	
11	17.02.2026	IMPLEMENTATION OF REAL TIME SIMULATION	34	
12	24.02.2026	SIMULATE A HYBRID SCHEDULER USING COMBINATION OF RMS AND EDF	37	
13	03.03.2026	WRITE A PROGRAM USING POSIX REAL-TIME THREADS	40	
14	10.03.2026	STUDY AND SIMULATE POWER MANAGEMENT APPROACHES USED IN ANDROID/IOS	43	
15	17.03.2026	IMPLEMENTATION OF ANDROID APPLICATIONS	46	

Ex. No.: 1 **SIMULATION OF PROCESS SCHEDULING**

Date: 02.12.2025

AIM

To simulate CPU process scheduling and calculate waiting time and turnaround time.

PROCEDURE

1. Enter the number of processes and burst times.
2. Apply First Come First Serve scheduling.
3. Calculate waiting time for each process.
4. Calculate turnaround time for each process.
5. Display the results and averages.

PROGRAM

```
n = int(input("Enter number of processes: ")) bt = [] wt = [0] *
n tat = [0] * n
for i in range(n):
    bt.append(int(input(f"Enter burst time for P{i+1}: ")))
for i in range(1, n):
    wt[i] = wt[i-1] + bt[i-1]
avg_wt = 0 avg_tat = 0
for i in range(n):
  tat[i] = wt[i] + bt[i]    avg_wt += wt[i]    avg_tat +=
    tat[i]
    print(f"P{i+1}\tBT={bt[i]}\tWT={wt[i]}\tTAT={tat[i]}")

print("Average Waiting Time =", avg_wt / n)
print("Average Turnaround Time =", avg_tat / n)
```

OUTPUT

Practice 1 - Sample Output

python output

```
Enter number of processes: 3
Enter burst time for P1: 4
Enter burst time for P2: 6
Enter burst time for P3: 2

P1    BT=4    WT=0    TAT=4
P2    BT=6    WT=4    TAT=10
P3    BT=2    WT=10   TAT=12

Average Waiting Time = 4.67
Average Turnaround Time = 8.67
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No. :2 **IMPLEMENTATION OF SYSTEM CALLS**

Date: 09.12.2025

AIM

To study basic file handling system call behavior using Python.

PROCEDURE

1. Open a file in read mode.
2. Read the contents of the file.
3. Display the contents.
4. Close the file.

PROGRAM

```
try:
    with open("sample.txt", "r") as file:
        data = file.read()
 print("File Content:")
 print(data) except
 FileNotFoundError:
      print("sample.txt  not  found")
```

OUTPUT

Practice 2 - Sample Output

python output

```
File Content:
Advanced Operating System Lab
System Calls Demonstration
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No. : 3 **IMPLEMENTATION OF DEADLOCKS**

Date: 16.12.2025

AIM

To implement Banker's Algorithm for deadlock avoidance.

PROCEDURE

1. Define allocation, maximum, and available resource matrices.
2. Compute the need matrix.
3. Check whether the system is in a safe state.

PROGRAM

```
alloc = [[0, 1, 0], [2, 0, 0], [3, 0, 2]] maxm = [[7, 5, 3], [3, 2, 2], [9, 0, 2]] avail = [3, 3, 2]

n = len(alloc) m = len(avail)
need = [[maxm[i][j] - alloc[i][j]
for j in range(m)]:
      for i in range(n)]:
          finish = [False] * n
       safe_seq = []
while len(safe_seq) < n:
    found = False     for i in range(n):
 if not finish[i] and all(need[i][j] <= avail[j]
 for j in range(m)):
  for j in range(m):
          avail[j] +=
           alloc[i][j]
     safe_seq.append(i)
       finish[i] = True
       found = True
if not found:
    break

if len(safe_seq) == n:
    print("System is in safe
    state")
print("Safe sequence:", safe_seq)
    else: print("System is not in safe
    state")
```

OUTPUT

Practice 3 - Sample Output

python output

```
System is in safe state
Safe sequence: [1, 2, 0]
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

IMPLEMENTATION OF PAGE REPLACEMENT ALGORITHM

Ex. No. :4

Date: 23.12.2025

AIM

To implement FIFO page replacement algorithm and count page faults.

PROCEDURE

1. Enter the page reference string.
2. Enter the number of frames.
3. Load pages into frames using FIFO method.
4. Count and display page faults.

PROGRAM

```
pages = list(map(int, input("Enter page reference string: ").split()))
frames_count = int(input("Enter number of frames: "))
frames = [] index = 0
faults = 0

for page in pages:
    if page not in frames:
        if len(frames) < frames_count:
            frames.append(page)
    else:
        frames[index] = page
        index = (index + 1) %
    frames_count faults += 1
    print("Frames:", frames)
print("Total Page Faults =", faults)
```

OUTPUT

Practice 4 - Sample Output

python output

```
Enter page reference string: 1 2 3 2 4 1 5
Enter number of frames: 3

Frames: [1]
Frames: [1, 2]
Frames: [1, 2, 3]
Frames: [1, 2, 3]
Frames: [4, 2, 3]
Frames: [4, 1, 3]
Frames: [4, 1, 5]

Total Page Faults = 6
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No. :5 **IMPLEMENTATION OF SCHEDULING ALGORITHM**

Date: 06.01.2026

AIM

To implement Round Robin CPU scheduling.

PROCEDURE

1. Enter the number of processes and burst times.
2. Enter the time quantum.
3. Execute processes in circular order.
4. Compute waiting time and turnaround time.
5. Display the result.

PROGRAM

```
n = int(input("Enter number of processes: ")) bt =
[] for i in range(n):
    bt.append(int(input(f"Enter burst time for P{i+1}: ")))

quantum = int(input("Enter time quantum: "))
rem = bt[:] wt = [0] * n
time = 0

done = False while not done:
  done = True     for i in
  range(n): if rem[i] > 0:
  done = False
     if rem[i] > quantum:
    time += quantum
 rem[i] -= quantum
      else:
                time += rem[i]
              wt[i] = time -
              bt[i] rem[i] = 0

for i in range(n):
   tat = bt[i] + wt[i]
    print(f"P{i+1}  WT={wt[i]}  TAT={tat}")
```

OUTPUT

Practice 5 - Sample Output

python output

```
Enter number of processes: 3
Enter burst time for P1: 5
Enter burst time for P2: 3
Enter burst time for P3: 1
Enter time quantum: 2

P1 WT=4 TAT=9
P2 WT=5 TAT=8
P3 WT=4 TAT=5
```

RESULT

The program uses a while loop to calculate the sum of the series 1+2+3+...+10 and runs successfully.

Ex. No. :6 **IMPLEMENTATION OF FILE ACCESS METHODS**

Date: 13.01.2026

AIM

To implement sequential file access in Python.

PROCEDURE

1. Create a file.
2. Write some records into the file.
3. Open the file in read mode.
4. Read and display the records sequentially.

PROGRAM

```
with open("data.txt", "w") as file:
file.write("Advanced OS Lab\n")
    file.write("File Access Methods\n")

with open("data.txt", "r") as file:
 print(file.read())
```

OUTPUT

Practice 6 - Sample Output

python output

```
Advanced OS Lab
File Access Methods
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No.7 IMPLEMENTATION OF DISTRIBUTED SCHEDULING

Date: 20.01.2026

AIM

To simulate distributed scheduling among processors.

PROCEDURE

1. Define tasks and processor loads.
2. Assign each task to the least loaded processor.
3. Update the load values.
4. Display final allocation. Program (Python)

PROGRAM

```
tasks = [5, 2, 7, 1, 4]
load = [0, 0, 0]

for i, task in enumerate(tasks):
min_processor =
load.index(min(load))
load[min_processor] += task
    print(f"Task {i+1} assigned to Processor {min_processor+1}")
 for i in range(len(load)):
  print(f"Processor {i+1} Load = {load[i]}")
```

OUTPUT

Practice 7 - Sample Output

python output

```
Task 1 assigned to Processor 1
Task 2 assigned to Processor 2
Task 3 assigned to Processor 3
Task 4 assigned to Processor 2
Task 5 assigned to Processor 1

Processor 1 Load = 9
Processor 2 Load = 3
Processor 3 Load = 7
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No. :8 **IMPLEMENTATION OF CLOCK SYNCHRONIZATION**

Date: 27.01.2026

AIM

To simulate clock synchronization in distributed systems.

PROCEDURE

1. Define server and client clock values.
2. Compare client clocks with server clock.
3. Adjust client clocks.
4. Display values before and after synchronization

PROGRAM

```
server = 100
clients = [95, 97, 103]
print("ServerClock=, server)
for i in range(len(clients)):
        print(f"Client {i+1} before sync ={clients[i]}")
clients[i] = server
print(f"ClienT{i+1}after synC{clients[i]}")
```

OUTPUT

Practice 8 - Sample Output

python output

```
Server Clock = 100
Client 1 before sync = 95
Client 1 after sync = 100
Client 2 before sync = 97
Client 2 after sync = 100
Client 3 before sync = 103
Client 3 after sync = 100
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

.

IMPLEMENTATION OF CONCURRENCY CONTROL ALGORITHMS

Ex. No. :9

Date: 03.02.2026

AIM

To demonstrate mutual exclusion using Peterson's Algorithm.

PROCEDURE

1. Initialize flag variables and turn variable.
2. Allow process 1 to request entry.
3. Allow process 2 to request entry.
4. Observe that only one process enters the critical section at a time. Program (Python)

PROGRAM

```
flag = [False, False] turn = 0

def process(i):
    global turn    j = 1 -
    i flag[i] = True
    turn = j
while flag[j] and turn == j:
    pass
    print(f"Process {i} is in critical section")
     flag[i] = False
 process(0)  process(1)
```

OUTPUT

Practice 9 - Sample Output

python output

```
Process 0 is in critical section
Process 1 is in critical section
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

WRITE A PROGRAM USING POSIX REAL-TIME THREAD

Ex. No. :10

Date:10.02.2026

AIM

To create and execute threads concurrently in Python.

PROCEDURE

1. Import the threading module.
2. Define a thread function.
3. Create multiple threads.
4. Start and join the threads.
5. Display the execution messages.

PROGRAM

```
import threading

 def task(name):
   print(f"Thread
 {name} is running")

 t1=threading.Threa
 d(target=task,args=(
 1,)) t2 =
 threading.Thread(ta
 rget=task,
 args=(2,))

 t1.start() t2.start()
 t1.join()  t2.join()
```

OUTPUT

Practice 10 - Sample Output

python output

```
Thread 1 is running
Thread 2 is running
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified

Ex. No. :11 **IMPLEMENTATION OF REAL TIME SIMULATION**

Date: 17.02.2026

AIM

To simulate periodic real-time task execution.

PROCEDURE

1. Define time intervals.
2. Execute tasks based on their periods.
3. Display which task runs at each time instant.
4. Observe the schedule.

PROGRAM

```
for time in range(10):
    if time % 2 == 0:
        print(f"Time {time}: Task Aexecuted")
    if time % 5 == 0:
        print(f"Time {time}: Task B executed")
```

OUTPUT

Practice 11 - Sample Output

python output

```
Time 0: Task A executed
Time 0: Task B executed
Time 2: Task A executed
Time 4: Task A executed
Time 5: Task B executed
Time 6: Task A executed
Time 8: Task A executed
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

SIMULATE A HYBRID SCHEDULER USING COMBINATION OF RMS AND EDF

Ex. No. :12

Date: 24.02.2026

AIM

To simulate a hybrid scheduler using RMS and EDF concepts.

PROCEDURE

1. Define tasks with period and deadline.
2. Select task with smallest period using RMS.
3. Use deadline as tie-breaker using EDF.
4. Display the selected task.

PROGRAM

```
tasks = [
    {"id": 1, "period": 4, "deadline": 4},
    {"id": 2, "period": 5, "deadline": 3},
 {"id": 3, "period": 8, "deadline": 6}
]

selected = min(tasks, key=lambda x: (x["period"],
x["deadline"]))  print("Selected Task = T"  + str(selected["id"]))
```

OUTPUT

Practice 12 - Sample Output

python output

```
Selected Task = T1
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

WRITE A PROGRAM PRIORITY-BASED REAL-TIME THREAD SIMULATION

Ex. No. :13

Date: 03.03.2026

AIM

To simulate priority-based execution of real-time tasks using threads

PROCEDURE

1. Create a priority queue containing tasks and priorities.
2. Define a worker function to execute the queued tasks.
3. Start the worker thread.
4. Observe tasks executing in priority order.

PROGRAM

```
from queue import PriorityQueue
import threading
import time

tasks = PriorityQueue()
tasks.put((1, "High Priority
Task")) tasks.put((3,  "Low
Priority Task"))
tasks.put((2,  "Medium Priority Task"))

def worker():
    while not tasks.empty():
        priority, name = tasks.get()
        print(f"Executing {name} with priority {priority}")
        time.sleep(0.1)

thread = threading.Thread(target=worker)
thread.start()
thread.join()
```

OUTPUT

```
Program Output

>>> Program executed successfully
Executing High Priority Task with priority 1
Executing Medium Priority Task with priority 2
Executing Low Priority Task with priority 3
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

STUDY AND SIMULATE POWER MANAGEMENT APPROACHES USED IN ANDROID/IOS

Ex. No. :14

Date: 10.03.2026

AIM

To simulate basic mobile power management modes.

PROCEDURE

1. Enter battery level.
2. Check battery percentage range.
3. Assign a suitable power-saving mode
4. Display the current mode.

PROGRAM

```
battery = int(input("Enter battery percentage: "))

if battery > 50:
        print("Normal Mode")elif battery> 20:
                print("Optimized Power Saving
Mode") else:            print("Ultra Power Saving
Mode")
implementation"); C1 obj=new C1();
obj.display();
```

OUTPUT

Practice 14 - Sample Output

python output

```
Enter battery percentage: 18
Ultra Power Saving Mode
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.

Ex. No.: 15 **IMPLEMENTATION OF ANDROID APPLICATIONS**

Date: 17.03.2026

AIM

To demonstrate the logic of a simple Android-style application using Python.

PROCEDURE

1. Display a simple application message.
2. Simulate a button click event.
3. Execute the button function.
4. Display the output message.

PROGRAM

```
def button_click():
    print("Hello Android")

print("Simple Android Application Simulation") button_click()
```

OUTPUT

Practice 15 - Sample Output

python output

```
Simple Android Application Simulation
Hello Android
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.